PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

Dehallucination Add-On Module for Enhancing Reliability of AI Applications

Project ID	**39**
Version	**1.0 Approved**
Team	**Team of 3**

Member	Name	SRN
1	Abhinav Chugh	PES2UG24AM007
2	Aarush Nalwaya	PES2UG24AM006
3	Adarsh Teli	PES2UG24AM012

Prepared by: Project Team 39

Department of Computer Science and Engineering, PES University

Date created: 06 September 2026

Document Control

Field	Value
Document	Software Requirements Specification
Project ID	39
Project	Dehallucination Add-On Module for Enhancing Reliability of AI Applications
Course	Software Engineering — UE24CS341A
Institution	PES University, Bangalore
Version	1.0 Approved
Development Methodology	Agile
Prepared by	Abhinav Chugh; Aarush Nalwaya; Adarsh Teli
Primary Audience	Developers, testers, project evaluators, users and project stakeholders

Revision History

Name	Date	Reason For Changes	Version
Project Team 39	06-09-2026	Initial SRS prepared for Project ID 39	1.0

Source and scope note

This SRS follows the structure and terminology prescribed in the uploaded Jackfruit Software Engineering Project Guidelines. The selected project is Project ID 39, whose stated scope is to create a module that detects and reduces hallucinations in AI-generated text by cross-referencing facts with trusted sources or knowledge graphs, with integration into chatbots or content generators. The detailed requirements below are the project team's proposed, testable interpretation of that scope for implementation.

Table of Contents

1. Introduction 4
    1.1 Purpose 4
    1.2 Intended Audience and Reading Suggestions 4
    1.3 Product Scope 4
    1.4 References 5
2. Overall Description 6
    2.1 Product Perspective 6
    2.2 Product Functions 6
    2.3 User Classes and Characteristics 7
    2.4 Operating Environment 7
    2.5 Design and Implementation Constraints 7
    2.6 Assumptions and Dependencies 7
3. External Interface Requirements 8
    3.1 User Interfaces 8
    3.2 Software Interfaces 8
    3.3 Communications Interfaces 8
    3.4 Hardware Interfaces 8
    3.5 API Contract — Proposed 9
4. Analysis Models 10
    4.1 High-Level Use Case Model 10
    4.2 Main Verification Flow 10
    4.3 Data Model 11
5. System Features 12
    5.1 Request Ingestion and Validation 12
    5.2 Claim Extraction and Normalization 12
    5.3 Evidence Retrieval 13
    5.4 Claim Verification and Contradiction Detection 13
    5.5 Confidence Scoring 14
    5.6 Grounded Response Generation 14
    5.7 Verification Report and Audit Trail 14
    5.8 Configuration and Administration 15
6. Other Nonfunctional Requirements 16
    6.1 Performance Requirements 16
    6.2 Safety Requirements 16
    6.3 Security Requirements 16
    6.4 Software Quality Attributes 16
    6.5 Business and Domain Rules 16
    6.6 Availability and Recovery 17
7. Other Requirements 18
    7.1 Database Requirements 18
    7.2 Internationalization and Encoding 18
    7.3 Legal, Ethical and Privacy Considerations 18
    7.4 Deployment and Maintenance Requirements 18
    7.5 Product Ownership / Team Functionality Allocation 18
Appendix A: Glossary 19
Appendix B: Field Layouts 20
Appendix C: Requirement Traceability Matrix 21
Appendix D: Requirement Summary 22

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional and nonfunctional requirements for Project ID 39, the Dehallucination Add-On Module for Enhancing Reliability of AI Applications. The module is intended to operate as a reliability layer around an AI-generated response: it receives generated text, identifies factual claims, retrieves supporting or contradicting evidence from configured trusted sources or a knowledge graph, evaluates the claims, and produces a grounded response or verification report.

This document establishes a common baseline for implementation, testing, architecture and later requirement traceability. Each requirement is uniquely identified so that it can be mapped to architecture/design elements, code and test cases in subsequent Jackfruit phases.

1.2 Intended Audience and Reading Suggestions

- **Developers:** use Sections 2–5 to understand system behavior, interfaces and feature boundaries.
- **Testers:** use the uniquely identified requirements in Section 5 and the quality constraints in Section 6 to derive test cases.
- **Project evaluators:** use the document to assess requirement completeness, feasibility and traceability.
- **Project stakeholders/users:** use Sections 1–3 to understand purpose, scope and expected interaction.
- **Project team members:** use Section 5 and Appendix C as the basis for dividing implementation work and tracking completion.

1.3 Product Scope

The product is a software add-on module that improves the reliability of AI-generated text by checking factual claims against evidence. It is designed to be integrated with a chatbot or content-generation application rather than replacing the host AI model. The core workflow is: accept AI output → identify verifiable claims → retrieve evidence → compare claims with evidence → classify support/contradiction/insufficient evidence → assign confidence → produce a grounded response and an audit trail.

In scope

- Accepting text or structured AI responses through a defined interface.
- Extracting and normalizing factual claims from the response.
- Retrieving evidence from configured trusted sources and/or a knowledge graph.
- Matching claims to evidence and identifying support, contradiction or insufficient evidence.
- Computing a verification confidence score for claims and the overall response.
- Flagging unsupported or contradictory claims.
- Generating a revised/grounded response that preserves useful content while marking or removing unsupported claims according to policy.
- Returning evidence references and a verification summary.
- Maintaining an audit record for requests, claims, evidence and decisions.

Out of scope

- Training a new foundation language model from scratch.
- Guaranteeing that every real-world fact can be verified.
- Replacing a domain expert for high-stakes decisions.
- Unrestricted scraping of arbitrary websites.

- Making legal, medical, financial or other professional decisions on behalf of users.

1.4 References

- PES University, Department of Computer Science and Engineering, *JACKFRUIT — Software Engineering Project Guidelines*, UE24CS341A, 2026.
- Project ID 39 description in the supplied project-selection guidelines: *Dehallucination Add-On Module for Enhancing Reliability of AI Applications*.
- Internal project artifacts to be produced in later phases: architecture/design specification, project plan, test plan, validation specification and requirement traceability updates.

2. Overall Description

2.1 Product Perspective

The proposed module is a self-contained reliability component positioned between a host generative-AI application and its user or downstream consumer. It can be invoked through an API or integrated as a library/service. The host application supplies the generated response and optional context; the module performs verification and returns a verification-enhanced response.

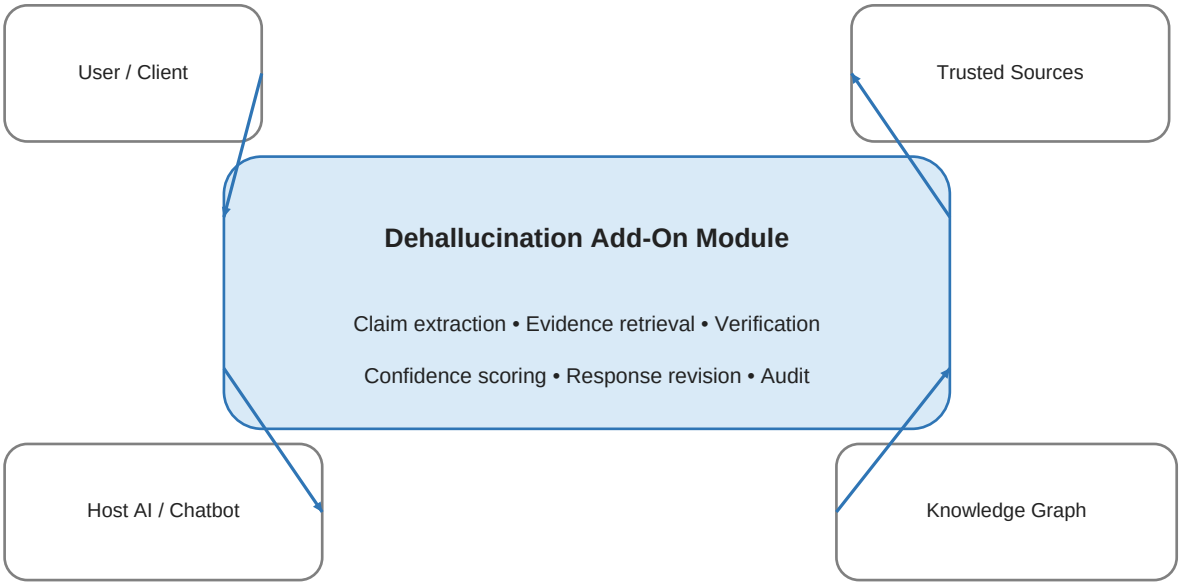


Figure 1. System context and major external interfaces

2.2 Product Functions

- Request ingestion and validation.
- Claim extraction and normalization.
- Evidence-source selection and retrieval.
- Evidence ranking and caching.
- Claim verification and contradiction detection.
- Confidence scoring and response-level reliability assessment.
- Grounded response generation with evidence references.
- Human-readable verification summary and audit logging.
- Configuration of trusted sources, thresholds and verification policies.
- Administrative review of verification history and system status.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Capabilities
End User / Consumer	May have little technical knowledge; consumes AI-generated answers.	Submit/use AI response; view grounded answer, warnings and evidence.
Host AI / Client Application	Software integrator calling the module programmatically.	Send requests; receive structured verification results; handle response status.
Developer / Integrator	Technical user responsible for integration and configuration.	Configure adapters, trusted sources, thresholds and API credentials.
Administrator / Reviewer	Privileged user monitoring quality and audit information.	Review logs, verification outcomes, source status and configuration.

2.4 Operating Environment

- Server-side execution on a modern Linux/Windows development environment.
- Python-based backend service with an HTTP/JSON API is the proposed implementation environment.
- A relational database is proposed for persistent audit and configuration data.
- A vector/keyword retrieval component may be used for evidence search; the final implementation choice is an engineering decision.
- Docker is proposed for reproducible local deployment and CI execution.
- Web browser support is required only for the optional administrative/review interface.

2.5 Design and Implementation Constraints

- The system must be modular so that evidence connectors and verification strategies can be replaced independently.
- External source availability, rate limits and API policies can constrain verification.
- Only configured trusted sources may be treated as authoritative evidence.
- The system must distinguish 'not verified' from 'false'; absence of evidence must not automatically be treated as contradiction.
- Sensitive request content should not be written to logs unless required by the configured audit policy.
- The project must remain explainable to the development team; model-assisted components must have documented inputs, outputs and failure behavior.

2.6 Assumptions and Dependencies

- The host application provides a text response or a supported structured response.
- At least one trusted evidence source or knowledge graph is available for the target domain.
- External connectors may fail or return incomplete evidence; the module therefore needs explicit degraded-mode behavior.
- The final implementation may use third-party NLP/embedding libraries, subject to license and compatibility checks.
- Thresholds for confidence and claim risk are configurable and may be tuned during testing.

3. External Interface Requirements

3.1 User Interfaces

The primary interface is an API-oriented interface for host applications. An optional web interface is proposed for reviewers and developers. The UI should expose the original response, extracted claims, verification status, evidence references, confidence values and final grounded response without requiring the user to understand internal model details.

Screen / View	Required Elements
Verification Result	Original response, overall reliability status, confidence score, claim-level status, warnings, evidence references.
Claim Details	Claim text, evidence snippets/identifiers, verification decision, confidence, explanation.
Audit History	Request ID, timestamp, status, number of claims, verification summary, error/degraded-mode indicator.
Configuration	Trusted source list, verification threshold, response policy, connector status; restricted to authorized users.

3.2 Software Interfaces

Interface	Input	Output	Purpose
Host AI API	JSON request containing AI response and optional context	Verification result JSON	Integrate the module with chatbot/content generator.
Evidence Connector	Claim/query plus source configuration	Evidence items with source metadata	Retrieve supporting/contradicting information.
Knowledge Graph Connector	Normalized claim/entity/query	Graph facts/relationships	Provide structured evidence.
Database	Requests, claims, evidence metadata, verification results	Persisted records	Auditability and history.
Optional NLP/ML Components	Text/claim segments	Entities, embeddings, scores	Claim extraction, matching and ranking.

3.3 Communications Interfaces

- HTTP/HTTPS shall be used for service-to-service API communication.
- JSON shall be the default request and response representation.
- HTTPS shall be used for deployed environments where sensitive content or credentials are transmitted.
- Connector failures and timeouts shall be represented by explicit error/degraded-mode statuses rather than silent failures.
- API requests shall support a unique request identifier for correlation across logs and responses.

3.4 Hardware Interfaces

No dedicated hardware interface is required by the project scope. The system is intended to execute on general-purpose computing hardware capable of running the selected backend, retrieval and NLP components.

3.5 API Contract — Proposed

Field	Type	Required	Description
request_id	String	Yes	Unique identifier for a verification request.
input_text	String	Yes	AI-generated response to be checked.
context	String/Object	No	Optional user prompt, conversation context or metadata.
verification_policy	Object	No	Optional thresholds and output policy.
claims	Array	Output	Extracted factual claims.
evidence	Array	Output	Evidence records associated with claims.
overall_confidence	Number	Output	Aggregate reliability score in the configured range.
grounded_response	String	Output	Verified/revised response.
status	Enum	Output	completed / partially_verified / degraded / failed.

4. Analysis Models

4.1 High-Level Use Case Model

The primary use case is to verify an AI-generated response and return a grounded result. Secondary use cases cover configuration and audit review. The model below identifies the major actors and interactions without prescribing a particular UI implementation.

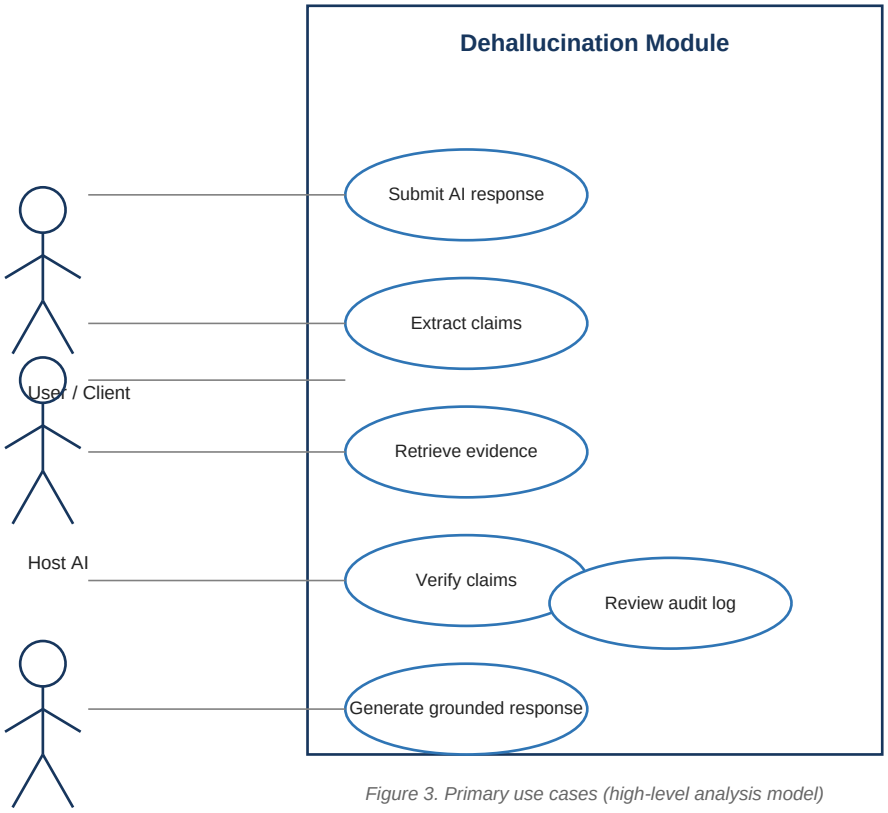


Figure 3. Primary use cases (high-level analysis model)

4.2 Main Verification Flow

Step	Actor/System Action	Result
1	Host AI submits generated response.	Request validated and assigned request ID.
2	Claim extractor analyzes response.	Candidate factual claims identified and normalized.
3	Risk classifier prioritizes claims.	Claims receive verification priority/risk level.
4	Evidence layer searches configured sources.	Evidence candidates returned and ranked.
5	Verification engine compares claim and evidence.	Each claim receives supported/contradicted/insufficient/uncertain decision.
6	Confidence scorer aggregates claim-level results.	Claim and response confidence calculated.
7	Response rewriter applies policy.	Grounded response and warnings generated.
8	Audit logger stores trace.	Request can be reviewed later.

4.3 Data Model

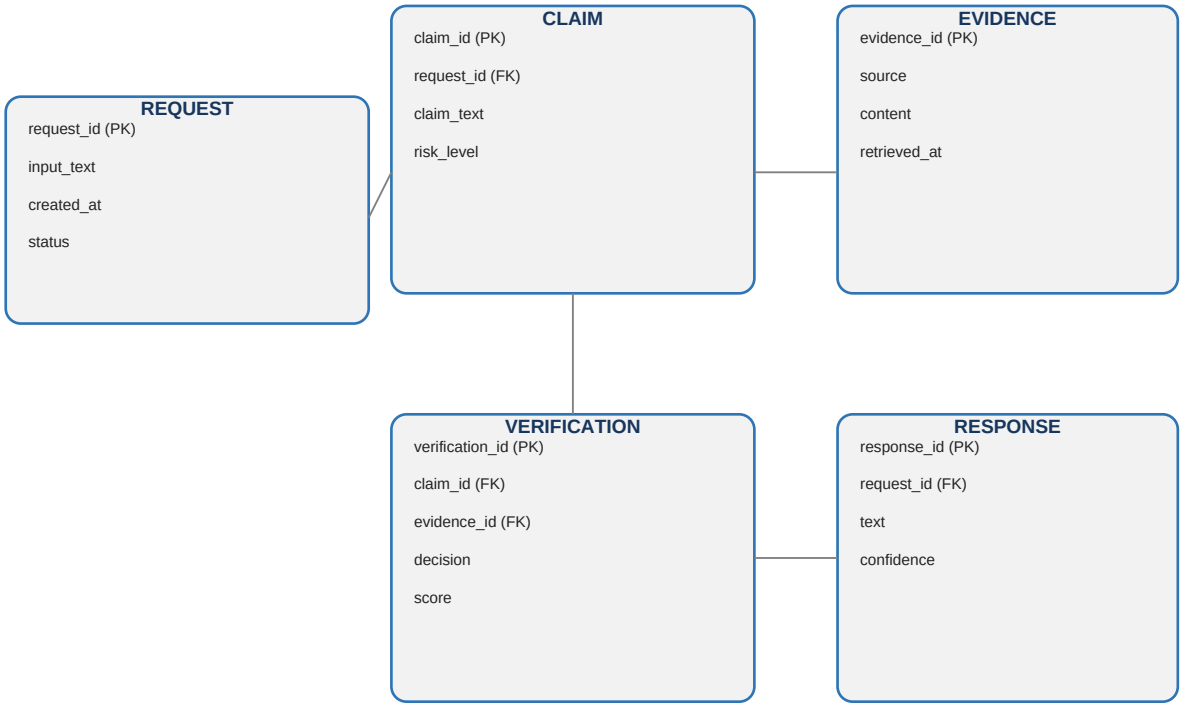


Figure 4. Logical data model for requests, claims, evidence and verification results

The logical data model separates the request, claims, evidence and verification decision so that individual claims can be independently traced to their supporting or contradicting evidence. The implementation may normalize these entities differently as long as the traceability relationships are preserved.

5. System Features

The requirements in this section are written as concise, testable capabilities. Priorities use High, Medium or Low as requested by the SRS template.

5.1 Request Ingestion and Validation

5.1.1 Description and Priority

Accept AI-generated content from the host application, validate the request structure and create a unique verification request. Priority: **High**.

5.1.2 Stimulus/Response Sequences

- Stimulus: host application submits a valid response.
- Response: system creates request ID and begins verification.
- Stimulus: required input is missing or invalid.
- Response: system returns a validation error without starting verification.

5.1.3 Functional Requirements

- **REQ-01:** The system shall accept a verification request containing a non-empty AI-generated text response.
- **REQ-02:** The system shall generate a unique request identifier for every accepted request.
- **REQ-03:** The system shall reject requests that do not contain required fields and shall return a machine-readable error.
- **REQ-04:** The system shall preserve the original submitted response without modifying it during ingestion.

5.2 Claim Extraction and Normalization

5.2.1 Description and Priority

The system shall identify factual claims that are candidates for verification and normalize them into a consistent internal representation. Priority: **High**.

5.2.2 Stimulus/Response Sequences

- Stimulus: valid request enters the analysis pipeline.
- Response: claims are extracted and assigned identifiers.
- Stimulus: response contains no verifiable claims.
- Response: system records zero claims and returns an appropriate status.

5.2.3 Functional Requirements

- **REQ-05:** The system shall segment the input response into candidate factual claims.
- **REQ-06:** The system shall assign a unique claim identifier to each extracted claim.
- **REQ-07:** The system shall retain the source text span from which each claim was extracted.
- **REQ-08:** The system shall classify claims as verifiable, non-verifiable/opinion-like, or uncertain.
- **REQ-09:** The system shall avoid presenting an opinion or instruction as a factual contradiction merely because external evidence is unavailable.

5.3 Evidence Retrieval

5.3.1 Description and Priority

Retrieve relevant evidence for each verifiable claim from configured trusted sources and/or a knowledge graph. Priority: **High**.

5.3.2 Stimulus/Response Sequences

- Stimulus: normalized claim is available.
- Response: system generates one or more evidence queries.
- Stimulus: source connector returns results.
- Response: evidence items are normalized and ranked.
- Stimulus: all connectors fail or return no usable evidence.
- Response: claim is marked insufficient/uncertain rather than false.

5.3.3 Functional Requirements

- **REQ-10:** The system shall query only configured evidence sources.
- **REQ-11:** The system shall associate each retrieved evidence item with its source identifier and retrieval timestamp.
- **REQ-12:** The system shall rank evidence candidates according to configured relevance criteria.
- **REQ-13:** The system shall support a knowledge-graph evidence connector as an implementation option.
- **REQ-14:** The system shall record when no sufficiently relevant evidence is found.
- **REQ-15:** The system shall enforce configurable connector timeout behavior.

5.4 Claim Verification and Contradiction Detection

5.4.1 Description and Priority

Compare each claim against retrieved evidence and classify the relationship. Priority: **High**.

5.4.2 Stimulus/Response Sequences

- Stimulus: claim and evidence candidates are available.
- Response: verification engine evaluates semantic agreement.
- Stimulus: evidence directly conflicts with the claim.
- Response: claim is flagged as contradicted.
- Stimulus: evidence is insufficient or ambiguous.
- Response: claim is marked uncertain/insufficient.

5.4.3 Functional Requirements

- **REQ-16:** The system shall compare a claim with its retrieved evidence.
- **REQ-17:** The system shall classify each verified claim into supported, contradicted, insufficient evidence, or uncertain.
- **REQ-18:** The system shall retain the evidence items used to make each claim-level decision.
- **REQ-19:** The system shall not classify a claim as contradicted solely because the retrieval stage returned no evidence.
- **REQ-20:** The system shall expose an explanation or decision rationale suitable for audit/review.

5.5 Confidence Scoring

5.5.1 Description and Priority

Calculate a confidence value that reflects the verification outcome and evidence quality. Priority: **High**.

5.5.2 Stimulus/Response Sequences

- Stimulus: claim-level decisions are available.
- Response: claim confidence scores are calculated.

- Stimulus: all claim scores are available.
- Response: overall response confidence is calculated using the configured aggregation policy.

5.5.3 Functional Requirements

- **REQ-21:** The system shall assign a confidence score to every claim that reaches verification.
- **REQ-22:** The system shall calculate an overall response reliability score from the claim-level results using a documented aggregation rule.
- **REQ-23:** The system shall expose the confidence score and its configured interpretation in the verification result.
- **REQ-24:** The system shall support configurable thresholds for warning or acceptance decisions.

5.6 Grounded Response Generation

5.6.1 Description and Priority

Produce a revised response that reduces unsupported claims while retaining supported information. Priority: **High**.

5.6.2 Stimulus/Response Sequences

- Stimulus: verification results are complete or partially complete.
- Response: response policy determines how flagged claims are handled.
- Stimulus: a claim is contradicted.
- Response: system removes, corrects or explicitly flags it according to configured policy.
- Stimulus: evidence is insufficient.
- Response: system marks uncertainty rather than fabricating support.

5.6.3 Functional Requirements

- **REQ-25:** The system shall generate a grounded response from the original response and verification results.
- **REQ-26:** The system shall not introduce a factual claim that is unsupported by the verification evidence available to the module.
- **REQ-27:** The system shall preserve supported content unless the configured response policy requires otherwise.
- **REQ-28:** The system shall mark or remove contradicted claims according to the configured policy.
- **REQ-29:** The system shall distinguish uncertain claims from verified claims in the returned result.
- **REQ-30:** The system shall provide evidence references for claims used in the grounded response where the selected output format supports references.

5.7 Verification Report and Audit Trail

5.7.1 Description and Priority

Provide a structured result and persistent audit information. Priority: **Medium**.

5.7.2 Stimulus/Response Sequences

- Stimulus: verification completes.
- Response: system returns a structured result containing status, claims, evidence and confidence.
- Stimulus: authorized reviewer requests history.
- Response: system returns permitted audit records.

5.7.3 Functional Requirements

- **REQ-31:** The system shall return the original request identifier in the verification result.
- **REQ-32:** The system shall return claim-level verification status and confidence.

- **REQ-33:** The system shall return evidence metadata sufficient to identify the source used.
- **REQ-34:** The system shall record verification status and timestamps for accepted requests.
- **REQ-35:** The system shall restrict audit information to authorized users or service identities.

5.8 Configuration and Administration

5.8.1 Description and Priority

Allow authorized users to configure evidence sources, thresholds and response policies. Priority: **Medium**.

5.8.2 Functional Requirements

- **REQ-36:** The system shall allow an authorized administrator to enable or disable configured evidence connectors.
- **REQ-37:** The system shall allow an authorized administrator to configure verification thresholds.
- **REQ-38:** The system shall validate configuration changes before activation.
- **REQ-39:** The system shall record configuration changes with timestamp and actor identity.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- **NFR-P01:** For a normal text request containing up to 20 verifiable claims, the system should return a result within 10 seconds when configured evidence sources respond within their timeout limits.
- **NFR-P02:** The system shall apply a configurable timeout to every external evidence connector.
- **NFR-P03:** Evidence retrieval should avoid repeated identical queries within the configured cache lifetime.
- **NFR-P04:** The service should support at least 10 concurrent verification requests in the initial prototype deployment without data corruption.

6.2 Safety Requirements

- **NFR-S01:** The system shall not represent an unverified claim as verified merely because a source is unavailable.
- **NFR-S02:** When verification is incomplete, the result shall explicitly indicate partial/degraded verification.
- **NFR-S03:** For potentially high-stakes content, the system shall display a verification limitation/disclaimer according to the configured policy.
- **NFR-S04:** The system shall fail safely: connector errors must not cause fabricated evidence or fabricated citations.

6.3 Security Requirements

- **NFR-SEC01:** Administrative configuration and audit endpoints shall require authentication.
- **NFR-SEC02:** Authorization shall prevent ordinary users from changing trusted-source configuration.
- **NFR-SEC03:** Secrets such as connector credentials shall not be stored in source code or ordinary request logs.
- **NFR-SEC04:** Data transmitted between clients and the deployed service shall use HTTPS.
- **NFR-SEC05:** Logs shall minimize exposure of user-provided content and shall support configurable retention.

6.4 Software Quality Attributes

Attribute	Requirement / Target
Correctness	Verification decisions shall be reproducible for the same inputs, evidence snapshot and configuration where deterministic components are used.
Reliability	External source failures shall result in explicit degraded status rather than silent success.
Maintainability	Evidence connectors, claim extraction, verification and response generation shall be separate modules with documented interfaces.
Testability	Core components shall expose unit-testable functions and integration boundaries.
Usability	Verification results shall clearly distinguish supported, contradicted and uncertain claims.
Portability	The service should run through a reproducible containerized setup.
Extensibility	New evidence sources and verification strategies should be addable without rewriting the request/API layer.
Auditability	Every final claim-level decision shall be traceable to the request and evidence used, when evidence exists.

6.5 Business and Domain Rules

- **BR-01:** Only configured trusted sources may be used to produce a 'supported' decision.
- **BR-02:** Evidence absence is not equivalent to contradiction.
- **BR-03:** A claim-level decision must remain linked to the evidence considered for that decision.
- **BR-04:** A response-level confidence score must not conceal the presence of a high-risk contradicted claim.
- **BR-05:** Administrative changes to verification policy must be auditable.

6.6 Availability and Recovery

- **NFR-A01:** The service shall return a meaningful error or degraded status when a dependency is unavailable.
- **NFR-A02:** Persistent audit data shall be protected using the selected database's backup/recovery mechanism.
- **NFR-A03:** The system shall not lose the request identifier when a downstream verification stage fails.

7. Other Requirements

7.1 Database Requirements

- The database shall store request metadata, extracted claims, evidence metadata, verification decisions and response metadata.
- Primary and foreign-key relationships shall preserve traceability from request → claim → evidence → verification decision.
- The implementation shall avoid storing raw sensitive content unless required for the project's audit objective.
- Indexes should be provided for request ID, claim ID, timestamps and verification status used in common queries.

7.2 Internationalization and Encoding

The initial prototype shall support UTF-8 text. The architecture should avoid assumptions that restrict claims or evidence to ASCII-only content.

7.3 Legal, Ethical and Privacy Considerations

- The module shall respect the terms, access policies and rate limits of external evidence sources.
- The system shall communicate that verification is evidence-based and not an absolute guarantee of truth.
- The design shall minimize retention of user-provided content and credentials.
- The project shall avoid presenting the system as a substitute for qualified professional review in high-stakes domains.

7.4 Deployment and Maintenance Requirements

- Source code shall be maintained in Git with separate branches or pull requests for substantial changes.
- Automated tests shall run in CI for every merge/pull request according to the team's project workflow.
- Containerization should be used to make the development and test environment reproducible.
- Static analysis and security validation should be included in the project pipeline as required by the course guidelines.
- Configuration values shall be externalized from application code.

7.5 Product Ownership / Team Functionality Allocation

The course guidelines require each member of a 3/4-person team to complete one full functionality. The following allocation provides clear ownership while still requiring all members to participate in the overall SDLC.

Member	Primary Functionality Ownership	Supporting Responsibilities	Qualitative Goal
Abhinav Chugh PES2UG24AM007	Claim Extraction & Verification Pipeline	API integration, unit tests, claim-level evaluation	Reliable claim identification and deterministic traceability.
Aarush Nalwaya PES2UG24AM006	Evidence Retrieval & Knowledge-Source Integration	Source configuration, retrieval testing, caching	Relevant evidence retrieval with explicit failure handling.
Adarsh Teli PES2UG24AM012	Grounded Response, Confidence & Audit Reporting	UI/reporting, system tests, security validation	Clear verification results and auditable response generation.

All three members are expected to contribute to requirements, architecture/design discussions, integration, code review, testing, documentation and final demonstration. The allocation identifies primary ownership rather than exclusive participation.

Appendix A: Glossary

Term	Definition
AI	Artificial Intelligence; computational systems capable of tasks such as language understanding or generation.
Hallucination	An AI-generated statement that is unsupported, inaccurate or contradicted by reliable evidence.
Dehallucination	The process of detecting, reducing or correcting unsupported AI-generated claims.
Claim	A statement in generated text that can potentially be checked against evidence.
Evidence	Information retrieved from a configured trusted source or knowledge graph and used to assess a claim.
Trusted Source	An evidence provider explicitly configured and accepted by the system administrator/project policy.
Knowledge Graph	A structured representation of entities and relationships that can provide machine-readable facts.
Verification	Comparison of a claim against available evidence to determine support, contradiction, insufficiency or uncertainty.
Grounded Response	A response whose factual claims are constrained by the evidence available to the verification module.
Confidence Score	A numeric value representing the system's assessed reliability of a claim or response under its configured policy.
Degraded Mode	A state in which verification is incomplete because one or more dependencies are unavailable or insufficient.
Audit Trail	Persistent information that allows a verification request and its decisions to be reviewed after processing.
Host AI	The chatbot or content-generation application that invokes the dehallucination module.
Connector	A software component that communicates with an external evidence source or knowledge graph.
SRS	Software Requirements Specification.
API	Application Programming Interface.

Appendix B: Field Layouts

The following proposed field layouts satisfy the template's requirement to document fields, properties/attributes and report requirements. They can be converted into the project's final Excel field-layout artifact in a later phase.

B.1 Verification Request Fields

Field	Length	Data Type	Description	Mandatory
request_id	36	String/UUID	Unique request identifier	Y
input_text	TBD	Text	AI-generated response to verify	Y
context	TBD	Text/JSON	Optional prompt/conversation context	N
policy_id	50	String	Verification policy identifier	N
created_at	Timestamp	DateTime	Request creation time	Y
status	30	Enum/String	Processing status	Y

B.2 Claim Fields

Field	Length	Data Type	Description	Mandatory
claim_id	36	String/UUID	Unique claim identifier	Y
request_id	36	String/UUID	Parent request identifier	Y
claim_text	TBD	Text	Normalized factual claim	Y
source_span	TBD	Text/JSON	Location in original response	Y
risk_level	15	Enum	Low/Medium/High/Uncertain	Y
verification_status	30	Enum	Supported/Contradicted/Insufficient/Uncertain	Y
confidence	Decimal	Numeric	Claim confidence score	Y

B.3 Evidence Fields

Field	Length	Data Type	Description	Mandatory
evidence_id	36	String/UUID	Unique evidence identifier	Y
claim_id	36	String/UUID	Claim being evaluated	Y
source_id	100	String	Configured source identifier	Y
source_reference	TBD	String	URL/document/graph reference as applicable	Y
content	TBD	Text	Evidence excerpt or structured fact	Y
retrieved_at	Timestamp	DateTime	Retrieval time	Y
relevance_score	Decimal	Numeric	Evidence relevance score	N

B.4 Verification Report Requirements

Verification Report	Fields
Summary Report	Request ID, timestamp, overall status, overall confidence, claim counts, warning count.
Claim Report	Claim ID, claim text, risk, decision, confidence, evidence references, rationale.
Evidence Report	Evidence ID, source ID, source reference, retrieval timestamp, relevance score.
Audit Report	Request ID, actor/service identity, timestamps, configuration/policy version, status and failure/degraded reason.

Appendix C: Requirement Traceability Matrix

The course template specifies traceability from requirement ID through architecture, design, code and tests. Architecture/design/code/test identifiers below are proposed placeholders to be finalized when the corresponding phase artifacts are created. They intentionally preserve traceability without inventing file names that do not yet exist.

Sl.	Requirement ID	Brief Description	Architecture Ref.	Design Ref.	Code Ref.	Test Case ID	System Test ID
1	REQ-01	Accept non-empty AI response	ARCH-ING-01	DES-API-01	TBD	UT-01	ST-01
2	REQ-02	Generate unique request ID	ARCH-ING-01	DES-API-02	TBD	UT-02	ST-01
3	REQ-05	Extract factual claims	ARCH-CLM-01	DES-CLM-01	TBD	UT-05	ST-02
4	REQ-08	Classify verifiable/non-verifiable claims	ARCH-CLM-02	DES-CLM-02	TBD	UT-08	ST-02
5	REQ-10	Query configured trusted sources	ARCH-EVD-01	DES-EVD-01	TBD	UT-10	ST-03
6	REQ-11	Record source metadata	ARCH-EVD-02	DES-EVD-02	TBD	UT-11	ST-03
7	REQ-17	Classify verification outcome	ARCH-VER-01	DES-VER-01	TBD	UT-17	ST-04
8	REQ-18	Link decisions to evidence	ARCH-VER-02	DES-VER-02	TBD	UT-18	ST-04
9	REQ-21	Claim confidence score	ARCH-CON-01	DES-CON-01	TBD	UT-21	ST-05
10	REQ-22	Overall response score	ARCH-CON-02	DES-CON-02	TBD	UT-22	ST-05
11	REQ-25	Generate grounded response	ARCH-OUT-01	DES-OUT-01	TBD	UT-25	ST-06
12	REQ-26	Do not introduce unsupported claim	ARCH-OUT-02	DES-OUT-02	TBD	UT-26	ST-06
13	REQ-31	Return request identifier	ARCH-REP-01	DES-REP-01	TBD	UT-31	ST-07
14	REQ-34	Record verification status/timestamp	ARCH-AUD-01	DES-AUD-01	TBD	UT-34	ST-07
15	REQ-36	Configure evidence connectors	ARCH-CFG-01	DES-CFG-01	TBD	UT-36	ST-08
16	NFR-P01	Target response time	ARCH-PERF-01	DES-PERF-01	TBD	PT-01	ST-09
17	NFR-S01	Do not mark unavailable evidence as verified	ARCH-SAFE-01	DES-SAFE-01	TBD	SEC-01	ST-10
18	NFR-SEC02	Restrict source configuration	ARCH-SEC-01	DES-SEC-01	TBD	SEC-02	ST-11
19	NFR-QA	Modular/testable architecture	ARCH-QA-01	DES-QA-01	TBD	QA-01	ST-12
20	BR-04	High-risk contradiction remains visible	ARCH-RULE-01	DES-RULE-01	TBD	BR-04	ST-13

Traceability completion rule

As implementation progresses, every requirement must be linked to the actual architecture component, design element, source-code file/module and corresponding unit/integration/system tests. 'TBD' entries must be replaced before the final report is submitted.

Appendix D: Requirement Summary

Category	Requirement IDs	Count
Request Ingestion	REQ-01 – REQ-04	4
Claim Extraction	REQ-05 – REQ-09	5
Evidence Retrieval	REQ-10 – REQ-15	6
Verification	REQ-16 – REQ-20	5
Confidence	REQ-21 – REQ-24	4
Grounded Response	REQ-25 – REQ-30	6
Reporting/Audit	REQ-31 – REQ-35	5
Configuration	REQ-36 – REQ-39	4
Total Functional Requirements	REQ-01 – REQ-39	39
Nonfunctional / Business Rules	NFR / BR	20+ targets

Acceptance-Oriented Summary

- A valid AI response can be submitted and receives a unique request ID.
- Factual claims are extracted and individually traceable.
- Claims are checked against configured trusted evidence sources and/or knowledge-graph facts.
- The system distinguishes supported, contradicted, insufficient and uncertain outcomes.
- Confidence is available at claim and response levels.
- The final output explicitly identifies uncertainty and does not fabricate evidence.
- Verification records are auditable and access-controlled.
- The architecture is modular enough to add or replace evidence connectors and verification strategies.

End of Software Requirements Specification — Project ID 39